API SYSTEMS

Task 1: Research Document

MONICA ARONG DARMANIN
INTERACTIVE MEDIA

Part 1 - RESTful APIs

Representational State Transfer, also known as REST. This is a type of API that allows systems to communicate over the internet using HTTP protocols. The web browser, mobile apps or other software acts like the client. They can communicate using RESTful APIs to interact with the server to create, read, update or delete resources. This also allows to send request and receiving responses from a server in formats such as JSON, XML or HTML (Codeacademy, 2025; Panwar, 2024).

Core principles

Client-Server Architecture

The client and server operate independently of each other, and they are responsible for the overall frontend experience and backend logic. Although they work together to complete tasks, they both have separate systems that allow them to progress independently without directly affecting one another (Codeacademy, 2025; Panwar, 2024).

Statelessness

The client's request to a server must contain all the necessary information for the process to work independently. The server does not have any session information about the client request. It does not rely on the server remembering previous interactions. This results in both the client and server can handle any message without needing to access earlier communications (Codeacademy, 2025).

Cacheability

The ability to store and reuse the responses must be specified in the server's responses. The client is allowed to reuse data for future requests if it is marked cacheable. This process allows for reducing unnecessary communication with the server to overall improve the performance of the system. For example, if a website uses the same header and footer, this effectively can store the cache of these images after the first request (Amazon Web Services, 2025; Panwar, 2024).

Uniform Interface

This ensures that the client and server communicate in a uniform and organised manner. Uniform Resource Identifiers, also known as URIs, are used to identify resources. This allows the clients to access them clearly and interact with the resources through representations such as JSON or XML. This also supports HATEOAS, in which clients use hyperlinks to navigate, improving the maintainability of the system (Amazon Web Services, 2025; Panwar, 2024).

Layered System

This approach allows intermediary layers to exist between the client and the server without the client being aware of these layers. These include caching, load balancing or other components. The client can connect to other authorised intermediaries between the client

and server in this layered system, and they can still receive requests to other servers when necessary. This works together to handle client requests while remaining transparent to the client (Amazon Web Services, 2025; Panwar, 2024).

How a Request to an API Works

The Client sends a request to a REST API endpoint URL using an HTTP method. The server processes the request and interacts with the resources, it returns a response in JSON accompanied by the HTTP status codes (Panwar, 2024):

- 200 OK: was successful.
- 201 Created: was created successfully.
- 204 No Content: was successful, but no content to return.
- 400 Bad Request: was invalid.
- 401 Unauthorised: must authenticate to access the resource.
- 404 Not Found: resource was not found.
- 500 Internal Server Error: An unexpected error occurred (Panwar, 2024).

HTTP Methods

RESTful APIs use standard HTTP methods to act as unique resources that correspond to their specific purpose (Panwar, 2024).

- GET: This fetches data from the server to get access to the resources with the specific URL.
- POST: This sends data to the server to create a new resource.
- PUT: This updates data to an existing resource.
- PATCH: This updates the resource partially.
- DELETE: This removes the resource.

QueryString Parameters and Request Body

Requests in RESTful APIs often contain parameters to control the data returned or sent. The primary purpose of query string parameters is that it is visible to the URL and are used for non-sensitive and dynamic data. This filters two special cases, which are searching and pagination. For example, the GET method only retrieves content from the specific author. When using POST, PUT or PATCH methods, the request body is used to carry more complex data, such as creating or updating resources. The DELETE method is used to remove the specific resource from the server (Ploesser, 2022).

Student Website Endpoints

I will be focusing on my PHP learning management system, specifically on the student side.

1. GET/students/profile – This retrieves the student's profile information.

2. GET/students/assignments – This returns the list of assignments that the student currently has.
3. GET/students/timetables - This returns the list of schedules that the student currently has.
4. GET/students/grades – This returns the list of grades and feedback.
5. GET/students/attendance – This allows the students to view their attendance records.
6. POST/students/enrolment – This allows students to enrol on a course or unit.
7. POST/students/assignments – This allows students to upload assignments (details and attachments).
8. PATCH/students/profile – This allows students to update certain information in their profile.
9. DELETE/students/assignments{id} – This allows students to remove an assignment they have uploaded.
10. DELETE/students/enrolment{id} – This will allow students to remove a unit that they are enrolled in.

Part 2 – Authorisation

OAuth 2.0 stands for Open Authorisation. This is a standard authorisation framework that allows a website or application to grant a user access to resources without sharing the user's credentials. The main benefit of OAuth 2.0 is its improved security, ensuring sensitive information is not directly exposed to third-party applications (Okta, n.d.).

Key Components of OAuth 2.0 Framework

Scope

- Determines the specific permission and level of access that the client's application is requesting.
- Ensures that the application only has access to the data necessary for the function.

Access Tokens

- A security piece of data issued by the authorisation server after user consent.
- They present granted permission and must be included to protect resources.

Client ID & Client Secret

- Client ID pinpoints the application requesting access and an endpoint URI to send access tokens or an authorisation code.
- Client Secret is a confidential key used to authenticate the client with the authorisation server when requesting tokens (Okta, n.d.).

Student Website Authorisation

- The Access Token is created when the student logs into the system. The website will send a request to the authentication server. If the login information is correct, the server will return a response to the client. The token would then be used in API requests to verify the student's identity.

- The Scope would define what actions the user is allowed to perform within the API. This ensures they can only access the resources that they are authorised to.

    - read: profile – view profile information.
    - read: assignments – view assignments.
    - submit: assignments – upload attachments.
    - read: grades – view grades and feedback.

- Client ID and Client Secret would be used to identify the application's request to access the API. The school website would act as the client, and the server would verify these credentials before issuing an access token.

Part 3 – Security Considerations

The Open Worldwide Application Security Project, known as OWASP. It is a foundational element and is a globally recognised list for critical security risks that target APIs. This project highlights common weaknesses to prioritise protection (OWASP, 2023).

1. Broken Object Level Authorisation - This lacks proper access controls and is considered the most critical API risk. It can allow attackers to access or modify resources belonging to the users. When APIs fail to verify that the authenticated user is authorised to access a specific object, it creates a wide attack surface. This results in sensitive data that can be exposed or changed.

2. Broken Authentication - This happens when authentication mechanisms are not implemented properly, allowing attackers to obtain passwords, tokens, or session keys. This results in gaining authorised access to a user's account, which can be temporarily or permanently taken over.

3. Broken Object Property Level Authorization - This may expose or enable modification to object properties that users should not have access to. The lack of proper authorisation and validation can allow attackers to exploit this by manipulating request parameters to read or update sensitive fields.

4. Unrestricted Resource Consumption - This highlights when an API does not properly limit the use of the system resources, such as CPU, memory and storage or other network scopes. Other resources integrate with external services such as emails, SMS or phone calls, and these service methods often acquire cost per request. Without proper controls and restrictions, attackers can abuse the API and potentially cause Denial of Service.

5. Broken Function Level Authorisation - This risk occurs when an API fails to properly enforce complex access controls with different hierarchies, groups or roles at the function or endpoint level. This allows attackers to exploit privileged operations or have access to administrative features without authorisation.

6. Unrestricted Access to Sensitive Business Flows - This risk exposes business processes, such as transactions, to insufficient controls. This makes it vulnerable because attackers can exploit these flows and potentially cause financial loss or reputational damage to the business.

7. Server Side Request Forgery - This occurs when an API retrieves remote resources without validating the user-supplied URI, enabling attackers to manipulate the application into sending unspecified requests to unexpected destinations. This request is made to internal to external systems by supplying malicious URLs to bypass firewalls or gather sensitive information, including Blind SSRS, where it is not returned.

8. Security Misconfiguration – When APIs are configured incorrectly, such as exposing unnecessary endpoints or failing to apply security patches. Attackers can often obtain unauthorised access to unprotected files or an insecure configuration. This may be vulnerable if it lacks proper security hardening or if unnecessary features are enabled.

9. Improper Inventory Management - This occurs when organisations fail to properly track and manage their API endpoints or versions left running unpatched using weak security requirements. Lack of documentation and retirement strategies can result in the leakage of sensitive data. Attackers can gain access to these and possibly take over the server.

10. Unsafe Consumption of APIs - This may risk when an application integrates with external or third-party APIs without properly validating or securing the interaction. Developers tend to trust these services, relying on weak controls for authentication, transport security or input validation. When a connected service is compromised, this can result in sensitive data exposure, which potentially on injects attacks or denial of service.

Student Website Security Considerations

- Broken Object Level Authorisation is one major risk to the system, and this happens when users have access to data that is not theirs. By making sure students data is protected, this would be verified through their access tokens. Every request will check its ownership of the requested resource and ensure only the appropriate users can view or modify their own data.

- Broken Authentication is another risk as it can allow attackers to gain access to users accounts. To reduce this risk, my API would ensure that it is secure with OAuth 2.0. This implementation helps secure authentication, enforcing strong passwords and expiring tokens. It reduces the risk of attackers gaining unauthorised access to student accounts

- Broken Object Property Level Authorization risk as attackers may manipulate request parameters to read or update sensitive fields. Students might attempt to access restricted fields. My API will enforce property-level permission, allowing the fields only for authorised users. This prevents the exposure or manipulation of sensitive data such as administrative data.

Reference list

Abhishek Panwar (2024). *REST API and Its Architecture*. [online] DEV Community. Available at: https://dev.to/abhishekpanwarrr/rest-api-and-its-architecture-3623

Amazon Web Services (2025). *What is RESTful API? - RESTful API Beginner's Guide - AWS*. [online] Amazon Web Services, Inc. Available at: https://aws.amazon.com/what-is/restful-api/.

Codecademy (2025). *What is REST API (RESTful API)? Explained*. [online] Codecademy. Available at: https://www.codecademy.com/article/what-is-rest-api.

Okta (n.d.). *What is OAuth 2.0, and what does it do for you?* [online] Auth0. Available at: https://auth0.com/intro-to-iam/what-is-oauth-2.

OWASP (2023). *OWASP Top 10 API Security Risks – 2023*. [online] owasp.org. Available at: https://owasp.org/API-Security/editions/2023/en/0x11-t10/.

Ploesser, K. (2022). *REST API Design Best Practices for Parameter and Query String Usage*. [online] API Product Management. Available at: https://www.moesif.com/blog/technical/api-design/REST-API-Design-Best-Practices-for-Parameters-and-Query-String-Usage/.